

Dokumentasi Proyek Ujian Akhir

Semester (UAS): Driver Drowsiness

Detection MobileNetV2

Dokumen ini memuat langkah-langkah komprehensif dari awal hingga akhir, dirancang untuk memfasilitasi mahasiswa Teknik Informatika dalam memahami alur kerja pengembangan model *Machine Learning*, mulai dari tahap persiapan dataset hingga cara menginterpretasikan hasil pelatihan model.

1. Pendahuluan

Proyek ini mengimplementasikan algoritma *Transfer Learning* dengan menggunakan pre-trained model **MobileNetV2**. Tujuannya adalah untuk mengklasifikasikan kondisi mata pengemudi (terbuka atau tertutup) guna mendeteksi gejala kantuk secara otomatis. Implementasi sistem ini merupakan bagian dari penerapan kecerdasan buatan dalam mencegah risiko kecelakaan lalu lintas.

2. Persiapan Lingkungan dan Dataset

Langkah pertama dalam *Jupyter Notebook* adalah mempersiapkan pustaka (libraries) dan mengelola dataset gambar.

1. **Import Library:** Memasukkan modul-modul esensial seperti TensorFlow/Keras untuk pembuatan model, OpenCV untuk manipulasi gambar, Pandas dan NumPy untuk komputasi array, serta Matplotlib untuk visualisasi grafik.
2. **Pengumpulan Dataset:** Menyiapkan kumpulan data (seperti dataset MRL Eye) yang memuat ribuan citra mata manusia dalam kondisi terbuka (*Open*) dan tertutup (*Closed*).
3. **Pemisahan Data:** Dataset dibagi menjadi subset pelatihan (*Training set*) dan subset pengujian/validasi (*Validation set*).

3. Pra-pemrosesan Data (Data Preprocessing)

Arsitektur MobileNetV2 membutuhkan spesifikasi data masukan yang konsisten. Tahapan ini

sangat krusial agar model dapat belajar dengan optimal.

- **Resize Gambar:** Semua citra pada dataset diubah resolusinya menjadi format 224 x 224 piksel.
- **Normalisasi:** Skala nilai piksel dikonversi dari rentang 0-255 menjadi 0-1 untuk mempercepat proses konvergensi model saat pelatihan.
- **Augmentasi Data:** Menerapkan *ImageDataGenerator* untuk memodifikasi gambar secara acak (rotasi ringan, zoom, pergeseran) agar model tidak menghafal data (mencegah *overfitting*).

4. Membangun Arsitektur Model MobileNetV2

Alih-alih melatih jaringan saraf tiruan dari nol, proyek ini memanfaatkan *Transfer Learning* dengan beban (weights) dari dataset ImageNet.

```
base_model = MobileNetV2(weights='imagenet', include_top=False, input_shape=(224, 224, 3))
base_model.trainable = False # Membekukan bobot dasar (Freezing)
```

1. **Membekukan Layer (Freezing):** Layer-layer konvolusi utama dari MobileNetV2 dibekukan agar bobot yang telah matang tidak berubah pada tahap awal pelatihan.
2. **Menambahkan Custom Head:** Menambahkan layer GlobalAveragePooling2D, layer Dropout (untuk reduksi overfitting), dan Dense layer terakhir dengan fungsi aktivasi sigmoid untuk klasifikasi biner.

5. Proses Pelatihan (Training Model)

Model yang telah disusun kemudian dikompilasi menggunakan optimizer **Adam**, fungsi loss **binary_crossentropy**, dan metrik pengukuran **accuracy**. Selanjutnya, model dilatih menggunakan data latih selama beberapa siklus (*epoch*), sembari dievaluasi kemampuannya menggunakan data validasi pada setiap akhir siklus.

6. Cara Membaca Data Hasil Training dari Dataset

Salah satu aspek terpenting dalam proyek *Machine Learning* adalah kemampuan membaca dan menginterpretasikan hasil pelatihan. Hasil ini biasanya divisualisasikan dalam bentuk grafik melalui Matplotlib.

A. Membaca Kurva Akurasi (Accuracy Curve)

Indikator Kurva	Interpretasi dan Cara Membaca
Training Accuracy (Akurasi Latih)	Garis ini menunjukkan persentase gambar dari dataset pelatihan yang berhasil ditebak dengan benar oleh model. Semakin mendekati 1.0 (100%), semakin baik model mengenali pola data yang dipelajarinya.
Validation Accuracy (Akurasi Validasi)	Garis ini krusial untuk mengukur performa di dunia nyata. Ini menunjukkan seberapa akurat model memprediksi gambar baru yang tidak pernah dilihat sebelumnya. Model yang ideal memiliki garis validasi yang berjalan beriringan dengan garis training.
Indikasi Overfitting	Jika garis Training Accuracy terus naik hingga mendekati 100%, tetapi Validation Accuracy tertahan (stagnan) di angka rendah atau bahkan menurun, berarti model hanya "menghafal" data dan gagal melakukan generalisasi.

B. Membaca Kurva Kerugian (Loss Curve)

- **Training Loss:** Mengukur besarnya error/kesalahan model saat belajar. Garis ini harus terus menunjukkan tren menurun secara perlahan menuju titik nol.
- **Validation Loss:** Mengukur error pada data uji. Apabila kurva Validation Loss tiba-tiba berbelok naik secara tajam di tengah-tengah pelatihan sementara Training Loss terus turun, proses pelatihan harus dihentikan (menggunakan *Early Stopping*) agar model tidak semakin memburuk.

C. Evaluasi Confusion Matrix

- **True Positives (TP):** Mata terdeteksi tertutup, dan kenyataannya memang tertutup (Alarm berhasil berbunyi).
- **False Negatives (FN):** Mata terdeteksi terbuka, padahal sebenarnya pengemudi sedang menutup mata. Ini adalah error paling berbahaya dalam sistem deteksi kantuk karena sistem gagal memberi peringatan.

7. Pengujian dan Implementasi Real-Time

Setelah mendapatkan metrik pelatihan yang memuaskan, model diekspor ke dalam format .h5. Pada tahap akhir, script implementasi menggunakan **OpenCV** akan mendeteksi wajah secara *real-time* menggunakan webcam. Bagian mata akan dipotong, dilewatkan pada model MobileNetV2, dan jika terprediksi tertutup, sistem memicu sirine alarm.

8. Interpretasi Detail & Representasi Grafik Pelatihan

(Catatan: Dokumen ini tidak dapat menyematkan file gambar (seperti .png atau .jpg) secara langsung dari output kode. Sebagai gantinya, data visual direpresentasikan secara komprehensif melalui tabel metrik di bawah ini, yang merepresentasikan pergerakan kurva pada Matplotlib/TensorBoard.)

Tabel Metrik Pelatihan (Representasi Kurva Epoch)

Tabel berikut memuat contoh data metrik log dari proses *training* yang membentuk grafik garis **Accuracy** dan **Loss** (Skenario Ideal):

Epoch	Training Accuracy	Validation Accuracy	Training Loss	Validation Loss
1	0.6502	0.6845	0.7850	0.7120
2	0.7410	0.7600	0.5620	0.5100
3	0.8105	0.8050	0.4100	0.4200

Epoch	Training Accuracy	Validation Accuracy	Training Loss	Validation Loss
4	0.8650	0.8500	0.3200	0.3400
5	0.9200	0.9150	0.1950	0.2100
10 (Final)	0.9650	0.9580	0.0980	0.1120

Analisis Representasi Kurva: Berdasarkan tabel di atas, seiring berjalannya epoch, nilai *Accuracy* (biru dan hijau) secara konsisten meningkat mendekati angka 1.0 (100%), sementara nilai *Loss* (merah dan oranye) terus menurun mendekati angka 0. Karena metrik Training dan Validation berjalan beriringan (tidak ada jarak/gap yang terlalu besar), model ini dikategorikan berhasil menggeneralisasi pola dan **terhindar dari overfitting**.